Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

Звіт про виконання лабораторної роботи №1
З дисципліни **«Функціональне програмування»**
На тему **«Основи функціонального програмування в python»**

Виконав:
Студент групи ФЕП-23
Литвинюк Ярослав

Львів 2026

Мета роботи:

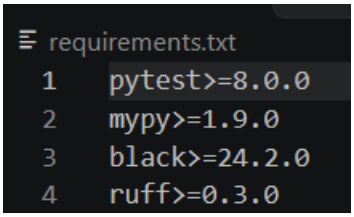
- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати typing.Callable для параметризації обчислень.

Варіант: №9

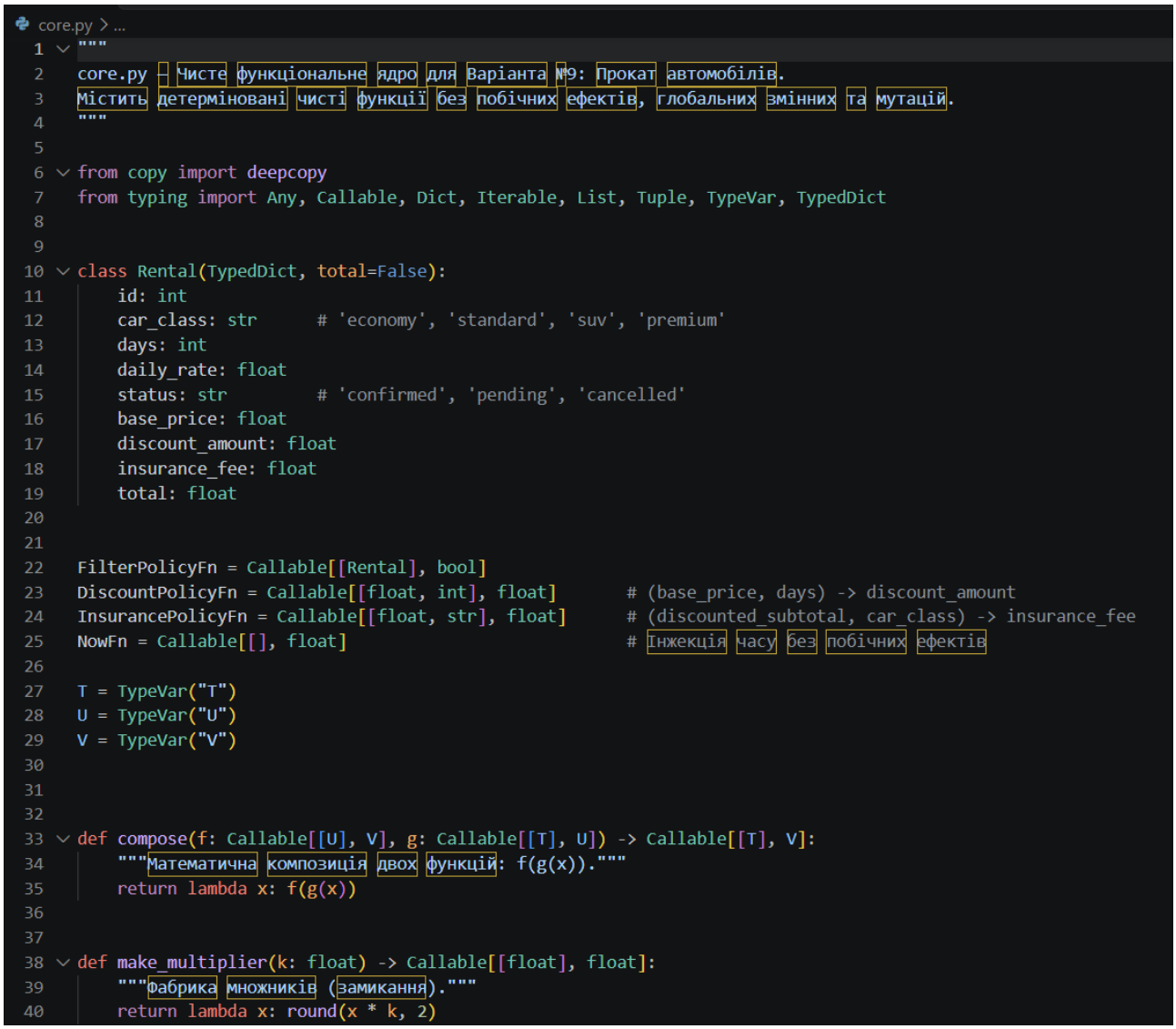
Хід роботи

```
lab01/
├── requirements.txt
├── core.py
├── app.py
├── README.md
└── tests/
    └── test_core.py
```

requirements.txt



core.py



```python
"""
core.py — Чисте функціональне ядро для Варіанта №9: Прокат автомобілів.
Містить детерміновані чисті функції без побічних ефектів, глобальних змінних та мутацій.
"""

from copy import deepcopy
from typing import Any, Callable, Dict, Iterable, List, Tuple, TypeVar, TypedDict


class Rental(TypedDict, total=False):
    id: int
    car_class: str      # 'economy', 'standard', 'suv', 'premium'
    days: int
    daily_rate: float
    status: str         # 'confirmed', 'pending', 'cancelled'
    base_price: float
    discount_amount: float
    insurance_fee: float
    total: float


FilterPolicyFn = Callable[[Rental], bool]
DiscountPolicyFn = Callable[[float, int], float]       # (base_price, days) -> discount_amount
InsurancePolicyFn = Callable[[float, str], float]      # (discounted_subtotal, car_class) -> insurance_fee
NowFn = Callable[[], float]                            # Інжекція часу без побічних ефектів

T = TypeVar("T")
U = TypeVar("U")
V = TypeVar("V")



def compose(f: Callable[[U], V], g: Callable[[T], U]) -> Callable[[T], V]:
    """Математична композиція двох функцій: f(g(x))."""
    return lambda x: f(g(x))


def make_multiplier(k: float) -> Callable[[float], float]:
    """Фабрика множників (замикання)."""
    return lambda x: round(x * k, 2)
```

```python
# --- Чисті функції предметної області (Варіант №9) ---

def calculate_base_price(rental: Rental) -> float:
    """Обчислює базову вартість оренди: дні * денна ставка."""
    return round(rental["days"] * rental["daily_rate"], 2)


def default_duration_discount(base_price: float, days: int) -> float:
    """
    Політика знижки за тривалістю:
    - від 7 днів: 15%
    - від 3 до 6 днів: 10%
    - менше 3 днів: 0%
    """
    if days >= 7:
        return round(base_price * 0.15, 2)
    if days >= 3:
        return round(base_price * 0.10, 2)
    return 0.0


def default_insurance_fee(subtotal: float, car_class: str) -> float:
    """
    Політика розрахунку обов'язкового страхового збору за класом авто:
    - premium: 15%
    - suv: 10%
    - economy / standard: 5%
    """
    rates = {
        "premium": 0.15,
        "suv": 0.10,
        "standard": 0.05,
        "economy": 0.05,
    }
    rate = rates.get(car_class.lower(), 0.05)
    return round(subtotal * rate, 2)
```

```python
def default_rental_filter(rental: Rental) -> bool:
    """Критерій валідності: статус 'confirmed' та тривалість більше 0 днів."""
    return rental.get("status") == "confirmed" and rental.get("days", 0) > 0


def with_rental_totals(
    rental: Rental,
    *,
    base_price: float,
    discount_amount: float,
    insurance_fee: float,
    total: float,
) -> Rental:
    """
    Повертає новий словник із розрахованими показниками,
    гарантуючи незмінність (immutability) вхідного об'єкта.
    """
    return {
        **rental,
        "base_price": base_price,
        "discount_amount": discount_amount,
        "insurance_fee": insurance_fee,
        "total": total,
    }


def stamp_record(record: Rental, now: NowFn) -> Tuple[Rental, float]:
    """Приклад ізоляції залежності від часу через інжекцію Callable."""
    return deepcopy(record), now()
```

```python
# --- Функція вищого порядку (Фабрика обробника) ---

def make_rental_processor(
    *,
    filter_policy: FilterPolicyFn = default_rental_filter,
    discount_policy: DiscountPolicyFn = default_duration_discount,
    insurance_policy: InsurancePolicyFn = default_insurance_fee,
) -> Callable[[Iterable[Rental]], Dict[str, Any]]:
    """
    Фабрика, що конструює чистий конвеєр обробки з інжектованими політиками.
    """
    def process(rentals: Iterable[Rental]) -> Dict[str, Any]:
        processed_rentals: List[Rental] = []
        total_revenue = 0.0
        total_discount = 0.0
        total_insurance = 0.0

        for r in rentals:
            if not filter_policy(r):
                continue

            base = calculate_base_price(r)
            discount = discount_policy(base, r["days"])
            subtotal = base - discount
            insurance = insurance_policy(subtotal, r["car_class"])
            final_total = round(subtotal + insurance, 2)

            updated = with_rental_totals(
                r,
                base_price=base,
                discount_amount=discount,
                insurance_fee=insurance,
                total=final_total,
            )
            processed_rentals.append(updated)

            total_revenue += final_total
            total_discount += discount
            total_insurance += insurance
```

```python
        return {
            "count": len(processed_rentals),
            "revenue": round(total_revenue, 2),
            "total_discounts": round(total_discount, 2),
            "total_insurance": round(total_insurance, 2),
            "rentals": processed_rentals,
        }

    return process


def process_rentals_pure(
    rentals: Iterable[Rental],
    *,
    filter_policy: FilterPolicyFn = default_rental_filter,
    discount_policy: DiscountPolicyFn = default_duration_discount,
    insurance_policy: InsurancePolicyFn = default_insurance_fee,
) -> Dict[str, Any]:
    """Прямий чистий виклик процесора замовлень."""
    processor = make_rental_processor(
        filter_policy=filter_policy,
        discount_policy=discount_policy,
        insurance_policy=insurance_policy,
    )
    return processor(rentals)
```

app.py

app.py > ...

```python
"""
app.py — Оболонка побічних ефектів (Imperative Shell).
Відповідає за консольний ввід/вивід (I/O), конфігурацію політик через Callable,
демонстрацію інжекції часу та виведення звітів.
"""

from datetime import datetime
import time
from typing import Any, Dict, List

from core import (
    Rental,
    make_rental_processor,
    process_rentals_pure,
    stamp_record,
)
```

```python
def render_report(
    result: Dict[str, Any], title: str = "ЗВІТ СИСТЕМИ ПРОКАТУ АВТОМОБІЛІВ"
) -> None:
    """
    Імперативна функція вводу/виводу (I/O).
    Форматує та виводить результати обчислень у консоль у вигляді таблиці.
    Споживає виключно чистий словник-результат без внесення мутацій.
    """
    print("=" * 82)
    print(f"{title:^82}")
    print("=" * 82)
    print(
        f"{'ID':<4} | {'Клас авто':<10} | {'Днів':<5} | {'Ставка':<8} | "
        f"{'Базова':<9} | {'Знижка':<8} | {'Страховка':<10} | {'Разом':<9}"
    )
    print("-" * 82)

    rentals: List[Rental] = result.get("rentals", [])
    for r in rentals:
        print(
            f"{r['id']:<4} | "
            f"{r['car_class'].capitalize():<10} | "
            f"{r['days']:<5} | "
            f"${r['daily_rate']:<7.2f} | "
            f"${r.get('base_price', 0.0):<8.2f} | "
            f"-${r.get('discount_amount', 0.0):<7.2f} | "
            f"+${r.get('insurance_fee', 0.0):<9.2f} | "
            f"${r.get('total', 0.0):<8.2f}"
        )

    print("-" * 82)
    print(f"Кількість підтверджених замовлень: {result['count']}")
    print(f"Сумарна надана знижка:             -${result['total_discounts']:.2f}")
    print(f"Сумарний страховий збір:           +${result['total_insurance']:.2f}")
    print(f"Загальний виторг компанії:          ${result['revenue']:.2f}")
    print("=" * 82)
```

```python
def main() -> None:
    # 1. Вхідний набір даних згідно зі специфікацією Варіанта №9
    raw_rentals: List[Rental] = [
        {
            "id": 101,
            "car_class": "economy",
            "days": 2,
            "daily_rate": 30.0,
            "status": "confirmed",  # 2 дні (<3) -> знижка 0%, страховка 5%
        },
        {
            "id": 102,
            "car_class": "standard",
            "days": 5,
            "daily_rate": 50.0,
            "status": "confirmed",  # 5 днів (3..6) -> знижка 10%, страховка 5%
        },
        {
            "id": 103,
            "car_class": "suv",
            "days": 4,
            "daily_rate": 85.0,
            "status": "cancelled",  # скасовано -> відсіюється фільтром
        },
        {
            "id": 104,
            "car_class": "suv",
            "days": 10,
            "daily_rate": 90.0,
            "status": "confirmed",  # 10 днів (>=7) -> знижка 15%, страховка 10%
        },
        {
            "id": 105,
            "car_class": "premium",
            "days": 7,
            "daily_rate": 180.0,
            "status": "confirmed",  # 7 днів (>=7) -> знижка 15%, страховка 15%
        },
        {
            "id": 106,
            "car_class": "economy",
            "days": 0,
            "daily_rate": 25.0,
            "status": "confirmed",  # 0 днів -> відсіюється фільтром
        },
        {
            "id": 107,
            "car_class": "premium",
            "days": 3,
            "daily_rate": 200.0,
            "status": "pending",  # pending -> відсіюється стандартним фільтром
        },
    ]

    # --- Демонстрація 1: Обробка за дефолтними політиками ядра ---
    default_report = process_rentals_pure(raw_rentals)
    render_report(default_report, title="СТАНДАРТНИЙ РОЗРАХУНОК (DEFAULT POLICIES)")

    # --- Демонстрація 2: Конфігурація кастомних політик через Callable ---
    # Приклад: промо-акція (Promo Weekend / VIP):
    # - підвищена знижка за тривалість (25% від 7 днів, 15% від 3 днів)
    def promo_discount_policy(base_price: float, days: int) -> float:
        if days >= 7:
            return round(base_price * 0.25, 2)
        if days >= 3:
            return round(base_price * 0.15, 2)
        return 0.0

    # - фіксована знижена страховка (7%) для всіх типів авто
    def flat_insurance_policy(subtotal: float, car_class: str) -> float:
        return round(subtotal * 0.07, 2)

    # - розширений критерій валідності: допускаємо і 'confirmed', і 'pending'
    flexible_filter = (
        lambda r: r.get("status") in {"confirmed", "pending"}
        and r.get("days", 0) > 0
    )
```

```
    promo_processor = make_rental_processor(
        filter_policy=flexible_filter,
        discount_policy=promo_discount_policy,
        insurance_policy=flat_insurance_policy,
    )
    promo_report = promo_processor(raw_rentals)
    print()
    render_report(
        promo_report, title="АКЦІЙНИЙ РОЗРАХУНОК (CUSTOM CALLABLE POLICIES)"
    )

    # --- Демонстрація 3: Інжекція недетермінованого часу (stamp_record) ---
    if default_report["rentals"]:
        first_record: Rental = default_report["rentals"][0]
        stamped_record, timestamp = stamp_record(first_record, time.time)
        formatted_time = datetime.fromtimestamp(timestamp).strftime(
            "%Y-%m-%d %H:%M:%S"
        )

        print()
        print(f"[Audit Log] Аудит замовлення ID #{stamped_record['id']}:")
        print(f"            Час фіксації:    {formatted_time} (UNIX: {timestamp:.4f})")
        print(f"            Підсумкова сума: ${stamped_record['total']:.2f}")


if __name__ == "__main__":
    main()
```

висновок: Опановано концепції чистих функцій, референтної прозорості та архітектурного шаблону *Functional Core / Imperative Shell*. Завдяки винесенню бізнес-політик у typing.Callable та відмові від мутацій досягнуто повної детермінованості обчислень та надійної ізоляції I/O-операцій.